PAGE REPLACEMENT ALGORITHM SIMULATOR

Operating Systems Lab Project Report

Name: Anuj Prajapati Roll No.: BTE25DTA000008

1. Aim

To implement and compare FIFO, LRU, and Optimal page replacement algorithms using Python in a Jupyter Notebook.

2. Background

Demand paging loads a page into main memory when it is needed. If the referenced page is not present, a page fault occurs. When no free frame remains, the operating system selects a resident page to replace. A hit occurs when the referenced page is already resident.

3. Algorithms

FIFO (First-In, First-Out)

Replaces the page that has been in memory the longest. It is straightforward to implement with a queue or circular pointer, but can exhibit Belady's anomaly.

LRU (Least Recently Used)

Replaces the page whose most recent access occurred farthest in the past. It uses past access history and requires additional recency bookkeeping.

Optimal

Replaces the page whose next use is farthest in the future, or which will never be used again. It produces the minimum fault count for a known reference string but cannot be implemented exactly online because future references are unknown.

4. Implementation

The program accepts a page reference string and a positive frame count. It validates the input, runs all three algorithms on the same reference sequence, and records the step number, page, frame contents, hit/fault status, and replaced page. It then calculates faults, hits, hit ratio, and fault ratio.

Hit ratio = page hits / total references. Fault ratio = page faults / total references.

5. Part B: Manual trace results

Reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2. Number of frames: 3.

Algorithm	Page faults	Page hits	Hit ratio

FIFO	10	3	23.08%
LRU	9	4	30.77%
Optimal	7	6	46.15%

6. Part C: Belady's anomaly

Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5. The simulator produced the following page-fault counts:

Algorithm	3 frames	4 frames
FIFO	9	10
LRU	10	8
Optimal	7	6

FIFO exhibits Belady's anomaly for this input: its page faults increase from 9 to 10 when the frame count increases from 3 to 4. LRU and Optimal have the stack property and do not exhibit this anomaly.

7. Custom experiment

LRU-better string (3 frames): 5 4 1 5 1 2 5 1 5 5 4 1 2 1 5 2 3 4. FIFO faults = 11; LRU faults = 9.

FIFO-equal string (4 frames): 5 1 5 3 5 2 1 5 5 2 3 1 5 1 5 1 5 2. FIFO faults = 4; LRU faults = 4.

8. Short questions

1. Why can Optimal not be implemented in a real operating system?

It requires exact knowledge of future page references, which a real operating system cannot know.

2. What extra bookkeeping does LRU need compared with FIFO?

LRU tracks recent use with timestamps, counters, or an ordered stack/list. FIFO mainly tracks arrival order.

3. If the number of frames equals the number of distinct pages, how many faults occur?

One fault per distinct page on its first reference; subsequent references hit because all distinct pages fit in memory.

9. Conclusion

The simulator demonstrates that replacement policies produce different fault counts for the same reference string. FIFO is simple but can show Belady's anomaly. LRU uses past access patterns, while Optimal is a lower-bound benchmark. The experiment reinforces the importance of replacement policy in virtual-memory management.

Before submission, add your name and roll number, execute the notebook from top to bottom, and attach genuine screenshots from your environment.